

DLFBT — Lab 1: de los apuntes a cada TO-DO

Cada apartado parte de la fórmula de los apuntes (amarillo), la transforma paso a paso y llega a la del lab (azul).

Los apuntes escriben la traspuesta como w^t ; aquí uso $w^\top$ para no confundirla con el target t .

Índices: $i = 1, \dots, N$ patrones $\cdot k = 1, \dots, d$ (o n_{l-1}) entradas $\cdot j = 1, \dots, n_l$ neuronas de la capa l . N patrones, d dimensión, η learning rate. Ej. 1, 2, 5: datos por **filas** (N, d). Ej. 6, 7: datos por **columnas** (d, N).

1. Regresión lineal (NumPy) — LinearRegressionModel

1.1. predict

```
y = np.dot(x, self.w) + self.b
```

APUNTES · Linear Regression · arbitrary dimension

$$y_i = w^t x_i + b$$

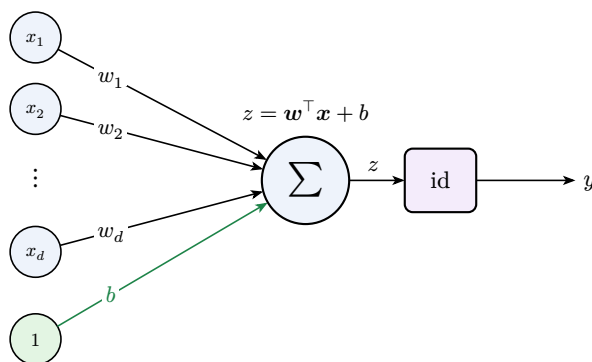
Apilo los N patrones como **filas** de una matriz. La fila i de Xw es $x_i^\top w = w^\top x_i$, así que las N predicciones salen de golpe:

$$X = \begin{pmatrix} x_1^\top \\ \vdots \\ x_N^\top \end{pmatrix} \Rightarrow \begin{pmatrix} y_1 \\ \vdots \\ y_N \end{pmatrix} = \begin{pmatrix} x_1^\top w + b \\ \vdots \\ x_N^\top w + b \end{pmatrix} = Xw + b$$

LAB

$$y = Xw + b$$

regresión lineal = una neurona con activación identidad



$$\begin{array}{c} \boxed{X} \\ N \times d \end{array} \cdot \begin{array}{c} \boxed{w} \\ d \times 1 \end{array} + \begin{array}{c} \boxed{b} \\ 1 \times 1 \end{array} = \begin{array}{c} \boxed{y} \\ N \times 1 \end{array}$$

El bias b (1×1) se suma a las N filas por *broadcasting*.

1.2. compute_gradients

```
delta = (y - t) / x.shape[0]
db = np.sum(delta, axis=0, keepdims=True)
dw = np.dot(x.T, delta)
```

APUNTES · Linear Regression · arbitrary dimension

$$E = \sum_{i=1}^N \frac{1}{2} (y_i - t_i)^2, \quad \nabla_{\mathbf{w}} E = \sum_{i=1}^N (y_i - t_i) \mathbf{x}_i, \quad \frac{\partial E}{\partial b} = \sum_{i=1}^N (y_i - t_i)$$

1. Suma → media. El lab usa la media (`0.5 * np.mean(...)`), no la suma: $L = E/N$, así que todo gradiente se divide entre N :

$$L = \frac{1}{2N} \sum_{i=1}^N (y_i - t_i)^2 \Rightarrow \nabla_{\mathbf{w}} L = \frac{1}{N} \sum_{i=1}^N (y_i - t_i) \mathbf{x}_i, \quad \frac{\partial L}{\partial b} = \frac{1}{N} \sum_{i=1}^N (y_i - t_i)$$

2. Agrupo el error. Llamo $\delta_i = (y_i - t_i)/N$ (es $\partial L / \partial y_i$):

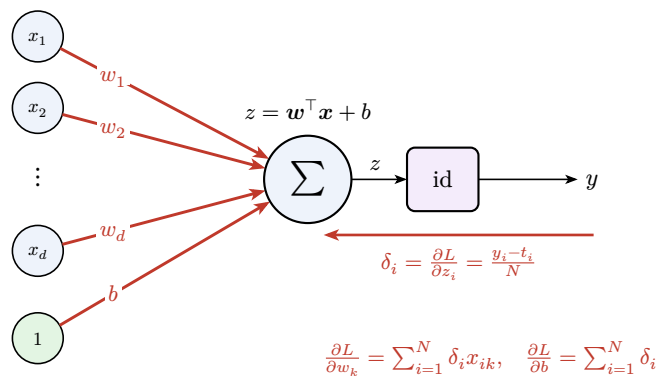
$$\nabla_{\mathbf{w}} L = \sum_{i=1}^N \delta_i \mathbf{x}_i, \quad \frac{\partial L}{\partial b} = \sum_{i=1}^N \delta_i$$

3. Suma → producto matricial. Las columnas de $\mathbf{X}^\top$ son los $\mathbf{x}_i$, y $\mathbf{X}^\top \boldsymbol{\delta}$ es la combinación de esas columnas con pesos δ_i :

$$\sum_{i=1}^N \delta_i \mathbf{x}_i = (\mathbf{x}_1 \dots \mathbf{x}_N) \begin{pmatrix} \delta_1 \\ \vdots \\ \delta_N \end{pmatrix} = \mathbf{X}^\top \boldsymbol{\delta}$$

LAB

$$\boldsymbol{\delta} = \frac{\mathbf{y} - \mathbf{t}}{N}, \quad \frac{\partial L}{\partial b} = \sum_{i=1}^N \delta_i, \quad \frac{\partial L}{\partial \mathbf{w}} = \mathbf{X}^\top \boldsymbol{\delta}$$



$$\begin{array}{c} \boxed{\mathbf{X}^\top} \\ d \times N \end{array} \cdot \begin{array}{c} \boxed{\boldsymbol{\delta}} \\ N \times 1 \end{array} = \begin{array}{c} \boxed{\partial L / \partial \mathbf{w}} \\ d \times 1 \end{array} \quad \begin{array}{c} \boxed{\boldsymbol{\delta}} \\ N \times 1 \end{array} \xrightarrow{\sum_{i=1}^N} \begin{array}{c} \boxed{\partial L / \partial b} \\ 1 \times 1 \end{array}$$

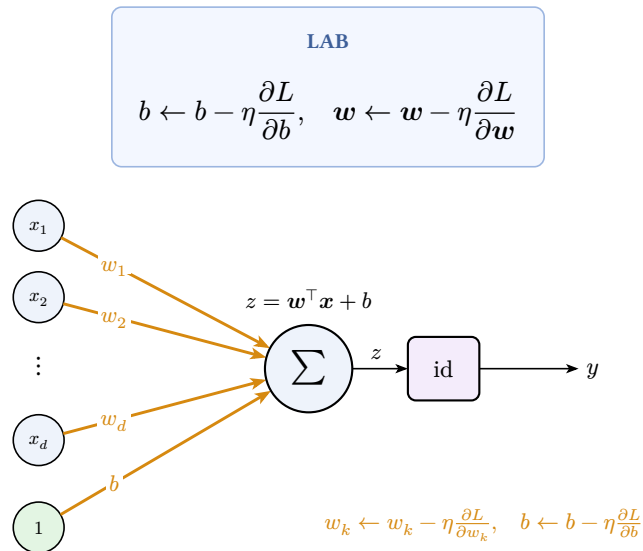
1.3. gradient_step

```
self.b = self.b - eta * db
self.w = self.w - eta * dw
```

APUNTES · Linear Regression · arbitrary dimension

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \nabla_{\mathbf{w}} E, \quad b_{t+1} = b_t - \eta \frac{\partial E}{\partial b}$$

| Idéntico, cambiando E por L (los gradientes del apartado anterior).



2. Regresión logística — LogisticRegressionModel

2.1. predict (sobrescrito)

```
def predict(self, x):  
    z = np.dot(x, self.w) + self.b  
    return LogisticRegressionModel.sigmoid(z)
```

APUNTES · Logistic Regression · arbitrary dimension

$$y_i = \sigma(\mathbf{w}^T \mathbf{x}_i + b), \quad \sigma(z) = \frac{e^z}{e^z + 1} = \frac{1}{1 + e^{-z}}$$

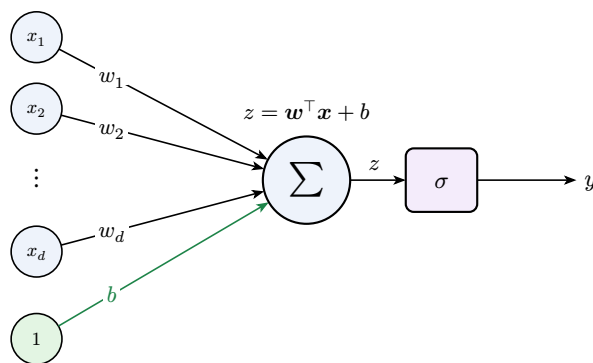
Es la regresión lineal seguida de σ elemento a elemento. Con el mismo apilado por filas que en el ej. 1:

$$\mathbf{z} = \mathbf{X}\mathbf{w} + b \Rightarrow \mathbf{y} = \sigma(\mathbf{z})$$

LAB

$$\mathbf{y} = \sigma(\mathbf{X}\mathbf{w} + b)$$

regresión logística = una neurona con activación sigmoide



$$\begin{array}{c} \boxed{X} \\ N \times d \end{array} \cdot \begin{array}{c} \boxed{w} \\ d \times 1 \end{array} + \begin{array}{c} \boxed{b} \\ 1 \times 1 \end{array} \xrightarrow{\sigma} \begin{array}{c} \boxed{y} \\ N \times 1 \end{array}$$

2.2. Gradientes (heredados, no hace falta sobrescribirlos)

```
# Heredado de LinearRegressionModel (no es un T0-D0 de esta clase):
delta = (y - t) / x.shape[0]
db = np.sum(delta, axis=0, keepdims=True)
dw = np.dot(x.T, delta)
```

APUNTES · Logistic Regression · cross-entropy / arbitrary dimension

$$C = - \sum_{i=1}^N [t_i \log y_i + (1 - t_i) \log(1 - y_i)], \quad \nabla_{\mathbf{w}} C = \sum_{i=1}^N (y_i - t_i) \mathbf{x}_i, \quad \frac{\partial C}{\partial b} = \sum_{i=1}^N (y_i - t_i), \quad \sigma'(z) = \sigma(z)(1 - \sigma(z))$$

Por qué sale lo mismo que en lineal: derivando respecto a la preactivación z_i con la regla de la cadena,

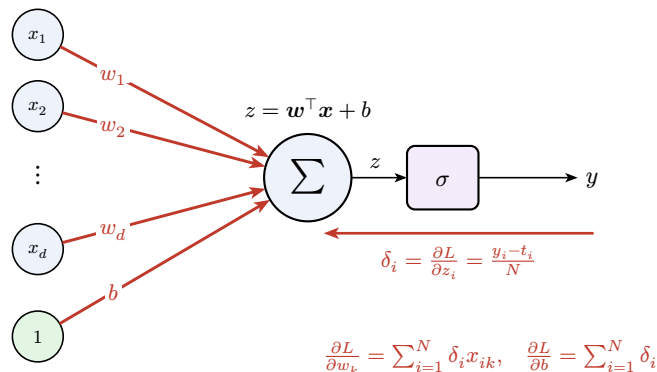
$$\frac{\partial C}{\partial y_i} = -\frac{t_i}{y_i} + \frac{1 - t_i}{1 - y_i} = \frac{y_i - t_i}{y_i(1 - y_i)}, \quad \frac{\partial y_i}{\partial z_i} = \sigma'(z_i) = y_i(1 - y_i)$$

$$\Rightarrow \frac{\partial C}{\partial z_i} = y_i - t_i$$

El lab usa la media ($L = C/N$, `-np.mean(...)`), así que $\partial L / \partial z_i = (y_i - t_i)/N = \delta_i$: exactamente la δ del ej. 1.

LAB

$$\delta = \frac{\mathbf{y} - \mathbf{t}}{N}, \quad \frac{\partial L}{\partial b} = \sum_{i=1}^N \delta_i, \quad \frac{\partial L}{\partial \mathbf{w}} = \mathbf{X}^\top \delta$$



`compute_gradients` y `gradient_step` se heredan de `LinearRegressionModel` sin cambios; sólo cambian `predict` y `get_loss`.

3. TensorFlow básico — BasicTF

3.1. differentiate

```
with tf.GradientTape() as tape:  
    y = f(x)  
dy_dx = tape.gradient(y, x).numpy()
```

APUNTES · Linear Regression · gradient descent

$$\frac{\partial E}{\partial w}$$

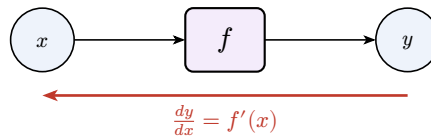
— los apuntes calculan las derivadas a mano; aquí las calcula TensorFlow.

f se aplica elemento a elemento: $y_m = f(x_m)$, con $m = 1, \dots, M$ recorriendo los M elementos de x .
`tape.gradient(y, x)` con y no escalar deriva la **suma** $\sum_{m=1}^M y_m$; como y_m sólo depende de x_m :

$$\frac{\partial}{\partial x_m} \sum_{m'=1}^M f(x_{m'}) = f'(x_m)$$

LAB

$$\frac{dy_m}{dx_m} = f'(x_m), \quad m = 1, \dots, M$$



`.numpy()` convierte el tensor resultado en array.

3.2. gradient_descent

```
with tf.GradientTape() as tape:  
    y = f(x)  
    dy_dx = tape.gradient(y, x)  
    x.assign(x - eta * dy_dx)
```

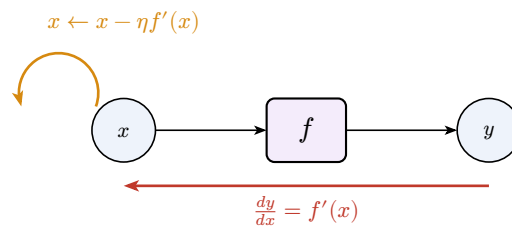
APUNTES · Linear Regression · gradient descent

$$w_1 = w_0 - \eta \frac{\partial E}{\partial w}$$

La variable a optimizar es x y la función a minimizar es $f: w \rightarrow x, E \rightarrow f$; el subíndice es la iteración, que llamo τ .

LAB

$$x_{\tau+1} = x_{\tau} - \eta f'(x_{\tau}) \quad (\tau = \text{iteración})$$



4. Regresión lineal (TensorFlow) — LinearRegressionModel_TF

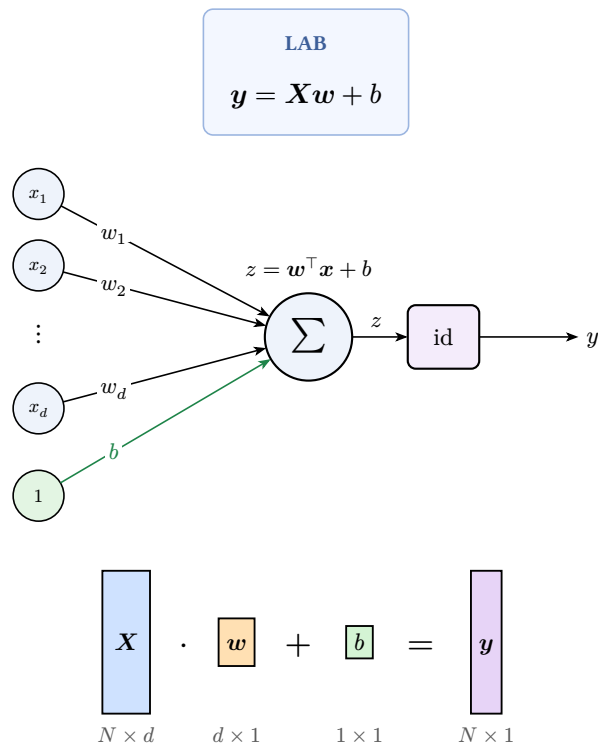
4.1. predict

```
y = tf.matmul(x, self.w) + self.b
```

APUNTES · Linear Regression · arbitrary dimension

$$y_i = w^t x_i + b$$

| Mismo apilado por filas que en el ej. 1; `tf.matmul` hace el papel de `np.dot`.



4.2. compute_gradients (autodiferenciación)

```
with tf.GradientTape() as tape:  
    loss = self.get_loss(x, t)  
db, dw = tape.gradient(loss, [self.b, self.w])
```

APUNTES · Linear Regression · gradient descent

$$E = \sum_{i=1}^N \frac{1}{2} (y_i - t_i)^2$$

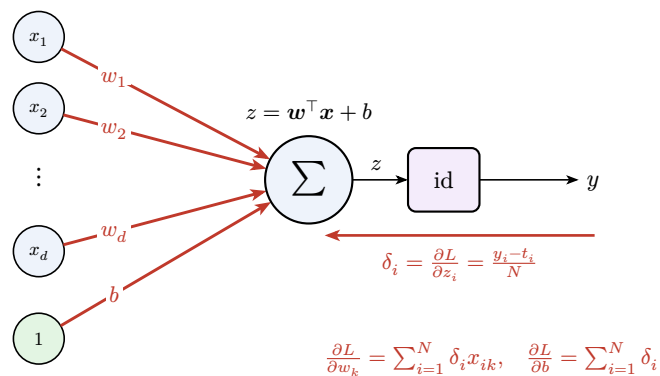
Con la media en vez de la suma (`tf.reduce_mean(0.5 * (y - t) * (y - t))`):

$$L = \frac{E}{N} = \frac{1}{2N} \sum_{i=1}^N (y_i - t_i)^2$$

No derivo a mano: la cinta graba las operaciones y aplica la regla de la cadena. Da lo mismo que el ej. 1: $\mathbf{X}^\top \boldsymbol{\delta}$ y $\sum_{i=1}^N \delta_i$.

LAB

$$\left(\frac{\partial L}{\partial b}, \frac{\partial L}{\partial \mathbf{w}} \right) = \text{tape.gradient}(L, [b, \mathbf{w}])$$

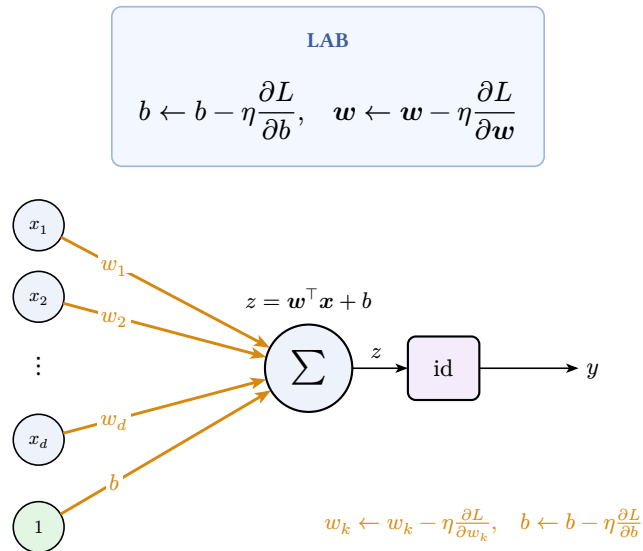


4.3. gradient_step

```
self.b.assign_sub(eta * db)
self.w.assign_sub(eta * dw)
```

APUNTES · Linear Regression · arbitrary dimension

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \nabla_{\mathbf{w}} E, \quad b_{t+1} = b_t - \eta \frac{\partial E}{\partial b}$$



`assign_sub(v)` hace `var = var - v` in-place sobre el `tf.Variable`.

5. Red neuronal (NumPy) — NeuralNetwork

Capas $l = 1, \dots, L$ con n_l neuronas; $\mathbf{y}^{(0)} = \mathbf{x}$. En el código la capa l es el índice $l-1$ de las listas.

5.1. predict (forward)

```
yl = x
for l in range(self.nlayers):
    zl = np.dot(self.W[l], yl) + self.b[l]
    yl = self.a[l](zl)
    z.append(zl)
    y.append(yl)
```

APUNTES · Non-linear models · artificial neuron, ec. (1) y (3) — índice $i \rightarrow k$ para no chocar con los patrones

$$z = \sum_{k=1}^d w_k x_k + b, \quad y = f(w^t x + b)$$

1. Una neurona j de la capa l . Su entrada es la salida de la capa anterior $\mathbf{y}^{(l-1)}$ y tiene sus propios pesos w_j y bias b_j :

$$z_j = \mathbf{w}_j^\top \mathbf{y}^{(l-1)} + b_j$$

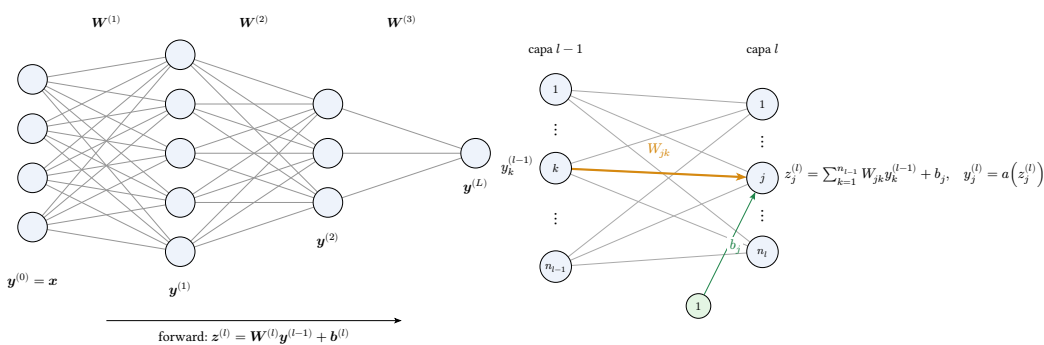
2. Toda la capa. Apilo los $\mathbf{w}_j^\top$ como filas de $\mathbf{W}^{(l)}$ ($n_l \times n_{l-1}$) y los b_j en $\mathbf{b}^{(l)}$:

$$\begin{pmatrix} z_1 \\ \vdots \\ z_{n_l} \end{pmatrix} = \begin{pmatrix} \mathbf{w}_1^\top \\ \vdots \\ \mathbf{w}_{n_l}^\top \end{pmatrix} \mathbf{y}^{(l-1)} + \begin{pmatrix} b_1 \\ \vdots \\ b_{n_l} \end{pmatrix} \implies \mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{y}^{(l-1)} + \mathbf{b}^{(l)}$$

3. Todo el batch. Aquí los patrones van como **columnas** ($d \times N$), así que cada columna se procesa igual y $\mathbf{b}^{(l)}$ se suma a las N columnas por broadcasting. La activación $f = a^{(l)}$ se aplica elemento a elemento.

LAB

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{y}^{(l-1)} + \mathbf{b}^{(l)}, \quad \mathbf{y}^{(l)} = a^{(l)}(\mathbf{z}^{(l)})$$



$$\begin{matrix} \boxed{\mathbf{W}^{(l)}} & \cdot & \boxed{\mathbf{y}^{(l-1)}} & + & \boxed{\mathbf{b}^{(l)}} & = & \boxed{\mathbf{z}^{(l)}} \\ n_l \times n_{l-1} & & n_{l-1} \times N & & n_l \times 1 & & n_l \times N \end{matrix}$$

5.2. compute_gradients (backpropagation)

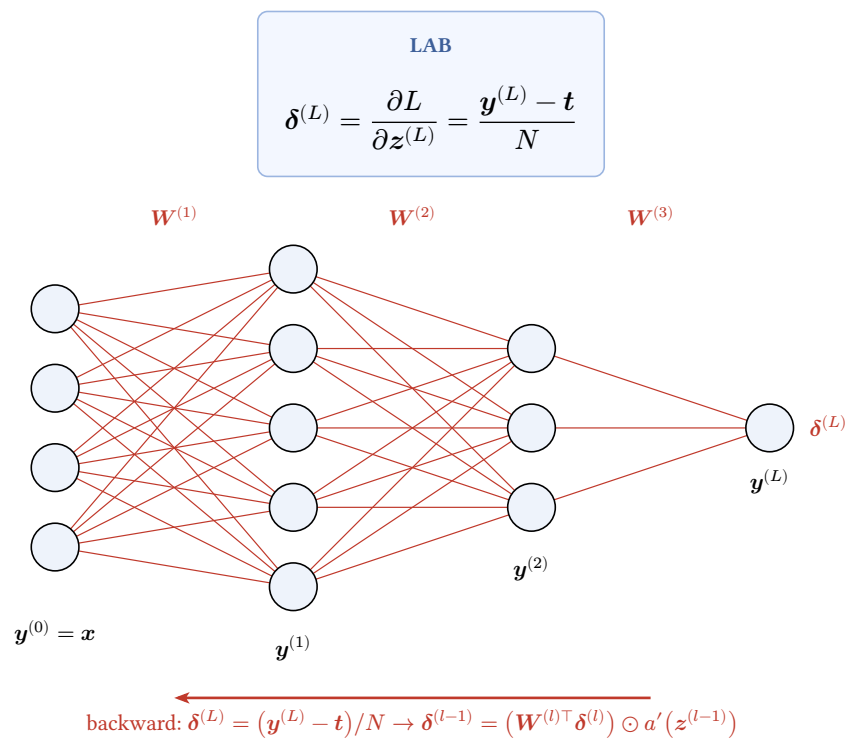
```
delta = dy
for l in range(self.nlayers - 1, -1, -1):
    y_prev = y[l-1] if l > 0 else x
    dW.insert(0, np.dot(delta, y_prev.T))
    db.insert(0, np.sum(delta, axis=1, keepdims=True))
    if l > 0:
        delta = np.dot(self.W[l].T, delta) * self.da[l-1](z[l-1])
```

5.2.1. Error en la última capa

APUNTES · Linear / Logistic Regression

$$\frac{\partial E}{\partial b} = \sum_{i=1}^N (y_i - t_i), \quad \frac{\partial C}{\partial b} = \sum_{i=1}^N (y_i - t_i)$$

Como $\partial z_i / \partial b = 1$, ambas fórmulas dicen que la derivada respecto a la preactivación es $y_i - t_i$ (lineal + MSE y sigmoide + cross-entropy; ver ej. 2). Dividiendo entre N por usar la media:



5.2.2. Gradientes de W y b

APUNTES · Linear Regression · arbitrary dimension

$$\nabla_{\mathbf{w}} E = \sum_{i=1}^N (y_i - t_i) \mathbf{x}_i$$

(error de la neurona $\times$ su entrada)

1. **Neurona j de la capa l .** Su «entrada» es $\mathbf{y}^{(l-1)}$ y su error $\delta_j^{(l)}$:

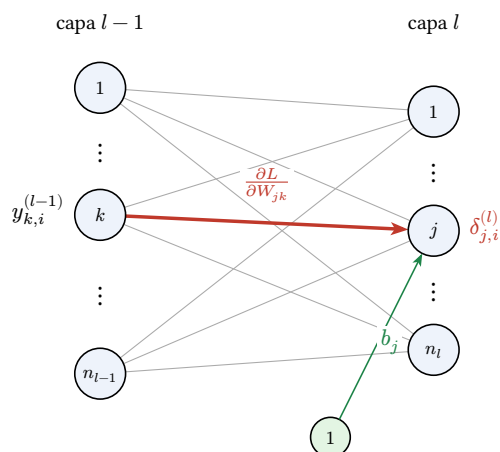
$$\nabla_{\mathbf{w}_j} L = \sum_{i=1}^N \delta_{j,i}^{(l)} \mathbf{y}_i^{(l-1)}, \quad \frac{\partial L}{\partial b_j} = \sum_{i=1}^N \delta_{j,i}^{(l)}$$

2. **Toda la capa.** Apilando por filas ($\mathbf{w}_j^\top$ es la fila j de $\mathbf{W}^{(l)}$), la fila j del resultado es $\sum_{i=1}^N \delta_{j,i}^{(l)} (\mathbf{y}_i^{(l-1)})^\top$; eso es un producto matriz $\times$ matriz traspuesta:

$$\frac{\partial L}{\partial \mathbf{W}^{(l)}} = \sum_{i=1}^N \delta_i^{(l)} (\mathbf{y}_i^{(l-1)})^\top = \boldsymbol{\delta}^{(l)} (\mathbf{y}^{(l-1)})^\top$$

LAB

$$\frac{\partial L}{\partial \mathbf{W}^{(l)}} = \boldsymbol{\delta}^{(l)} (\mathbf{y}^{(l-1)})^\top, \quad \frac{\partial L}{\partial \mathbf{b}^{(l)}} = \sum_{i=1}^N \delta_{:,i}^{(l)}$$



$$\frac{\partial L}{\partial w_{jk}} = \sum_{i=1}^N \delta_{j,i}^{(l)} y_{k,i}^{(l-1)}, \quad \frac{\partial L}{\partial b_j} = \sum_{i=1}^N \delta_{j,i}^{(l)}$$

$$\begin{matrix} \boxed{\boldsymbol{\delta}^{(l)}} & \cdot & \boxed{(\mathbf{y}^{(l-1)})^\top} & = & \boxed{\partial L / \partial \mathbf{W}^{(l)}} \\ n_l \times N & & N \times n_{l-1} & & n_l \times n_{l-1} \end{matrix}$$

$$\begin{matrix} \boxed{\boldsymbol{\delta}^{(l)}} & \xrightarrow{\sum_{i=1}^N} & \boxed{\partial L / \partial \mathbf{b}^{(l)}} \\ n_l \times N & & n_l \times 1 \end{matrix}$$

5.2.3. Propagar el error a la capa anterior

APUNTES · Logistic Regression · gradient descent

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

— los apuntes sólo tienen una neurona; esto es la regla de la cadena.

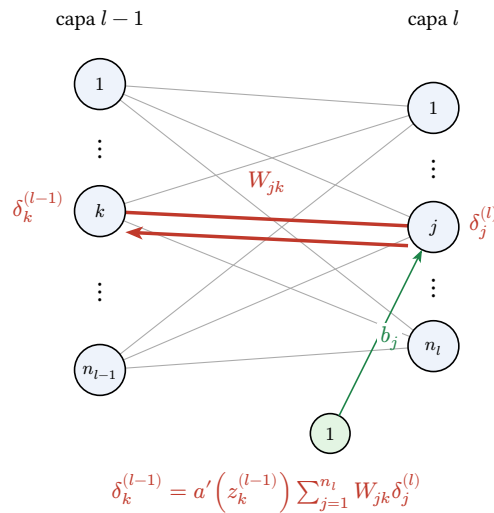
$z_k^{(l-1)}$ afecta a todas las neuronas j de la capa l a través de $y_k^{(l-1)} = f(z_k^{(l-1)})$ y del peso W_{jk} :

$$\delta_k^{(l-1)} = \sum_{j=1}^{n_l} \frac{\partial L}{\partial z_j^{(l)}} \frac{\partial z_j^{(l)}}{\partial y_k^{(l-1)}} \frac{\partial y_k^{(l-1)}}{\partial z_k^{(l-1)}} = f'(z_k^{(l-1)}) \sum_{j=1}^{n_l} W_{jk} \delta_j^{(l)}$$

$\sum_{j=1}^{n_l} W_{jk} \delta_j^{(l)}$ es la componente k de $\mathbf{W}^\top \boldsymbol{\delta}$; el factor f' va elemento a elemento:

LAB

$$\boldsymbol{\delta}^{(l-1)} = \left((\mathbf{W}^{(l)})^\top \boldsymbol{\delta}^{(l)} \right) \odot a'(z^{(l-1)})$$



$$\begin{array}{c} \boxed{(\mathbf{W}^{(l)})^\top} \\ n_{l-1} \times n_l \end{array} \cdot \begin{array}{c} \boxed{\boldsymbol{\delta}^{(l)}} \\ n_l \times N \end{array} \odot \begin{array}{c} \boxed{a'(z^{(l-1)})} \\ n_{l-1} \times N \end{array} = \begin{array}{c} \boxed{\boldsymbol{\delta}^{(l-1)}} \\ n_{l-1} \times N \end{array}$$

$\odot$ es el producto elemento a elemento (* en NumPy).

5.3. gradient_step

```
for l in range(self.nlayers):
    self.W[l] = self.W[l] - (eta * dW[l])
    self.b[l] = self.b[l] - (eta * db[l])
```

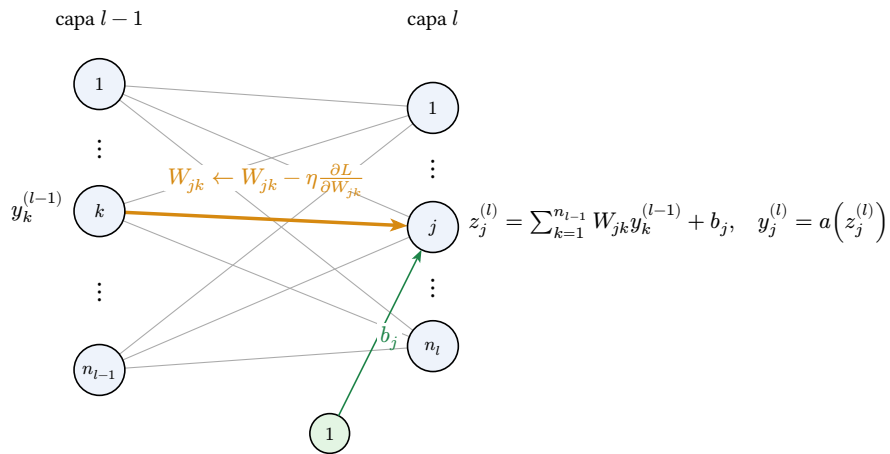
APUNTES · Linear Regression · arbitrary dimension

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \nabla_{\mathbf{w}} E, \quad b_{t+1} = b_t - \eta \frac{\partial E}{\partial b}$$

| La misma regla, aplicada a los parámetros de cada capa.

LAB

$$\mathbf{W}^{(l)} \leftarrow \mathbf{W}^{(l)} - \eta \frac{\partial L}{\partial \mathbf{W}^{(l)}}, \quad \mathbf{b}^{(l)} \leftarrow \mathbf{b}^{(l)} - \eta \frac{\partial L}{\partial \mathbf{b}^{(l)}} \quad \forall l$$



6. Red neuronal (TensorFlow) — NeuralNetwork_TF

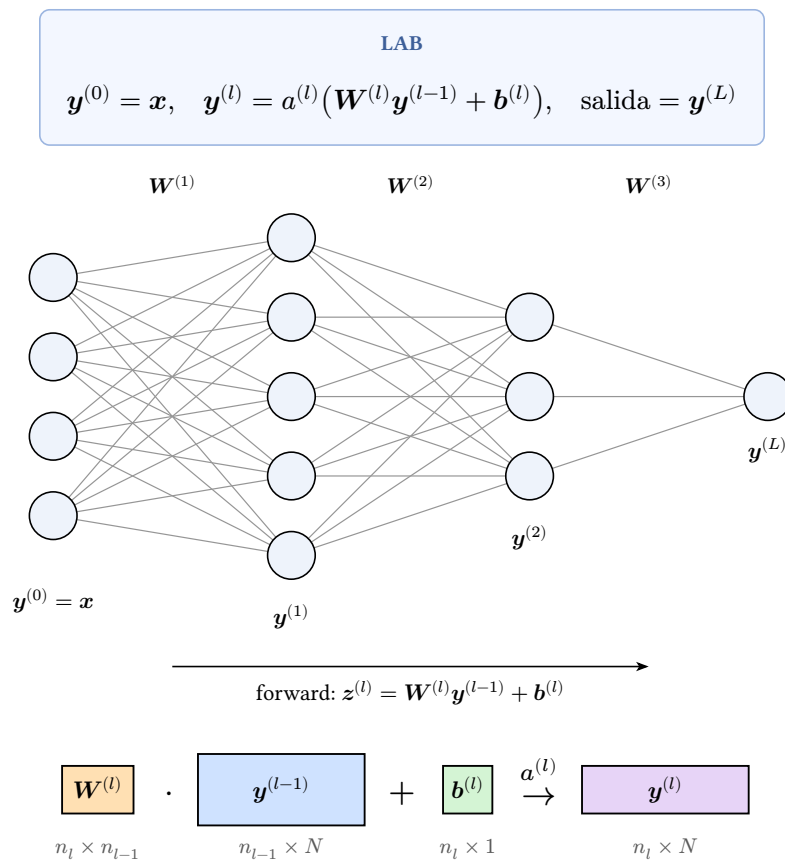
6.1. predict

```
y = x
for l in range(self.nlayers):
    z = tf.matmul(self.W[l], y) + self.b[l]
    y = self.a[l](z)
```

APUNTES · Non-linear models · artificial neuron, ec. (3)

$$y = f(w^t x + b)$$

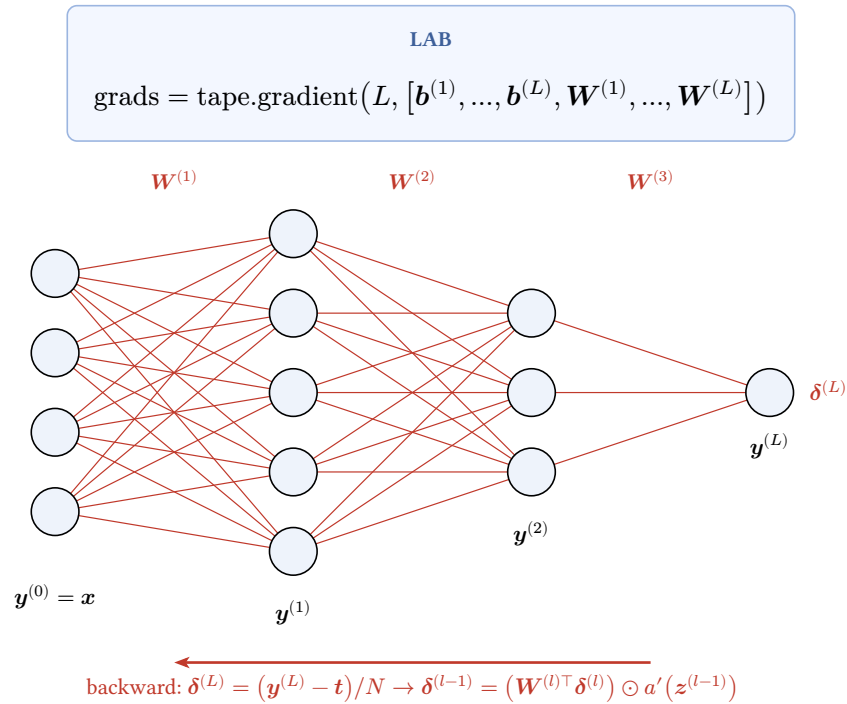
Igual que en el ej. 6 (apilado por capas y batch por columnas), encadenando las capas; sólo se devuelve la última activación.



6.2. compute_gradients

```
with tf.GradientTape() as tape:
    loss = self.get_loss(x, t, loss_function)
    grads = tape.gradient(loss, self.b + self.W)
    db = grads[:self.nlayers] # dL/db^(1..L)
    dW = grads[self.nlayers:] # dL/dW^(1..L)
```

La cinta hace el backprop del ej. 6 automáticamente, para todos los parámetros a la vez.



`self.b + self.W` concatena las dos listas de Python (no suma tensores), así que los gradientes vuelven en ese mismo orden: los primeros `nlayers` son los de los bias y el resto los de los pesos.

6.3. gradient_step

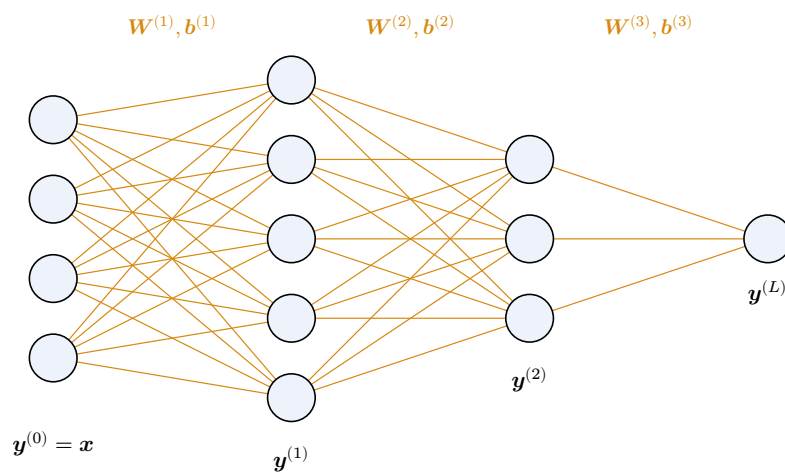
```
for l in range(self.nlayers):  
    self.b[l].assign_sub(eta * dB[l])  
    self.W[l].assign_sub(eta * dW[l])
```

APUNTES · Linear Regression · arbitrary dimension

$$w_{t+1} = w_t - \eta \nabla_w E, \quad b_{t+1} = b_t - \eta \frac{\partial E}{\partial b}$$

LAB

$$\mathbf{W}^{(l)} \leftarrow \mathbf{W}^{(l)} - \eta \frac{\partial L}{\partial \mathbf{W}^{(l)}}, \quad \mathbf{b}^{(l)} \leftarrow \mathbf{b}^{(l)} - \eta \frac{\partial L}{\partial \mathbf{b}^{(l)}} \quad \forall l$$



todas las $\mathbf{W}^{(l)}, \mathbf{b}^{(l)}$ se actualizan a la vez con su gradiente